

Face Recognition Using Eigenfaces

Andrea Parker

May 4, 2026

Abstract

Facial recognition serves as a widely used biometric identification technique because it operates efficiently and does not require intrusive data collection. In this work, we design and implement a face recognition system based on the Eigenfaces method using Principal Component Analysis (PCA). This approach reduces high-dimensional facial image data to a lower-dimensional subspace by extracting principal components that capture the most significant variations in the dataset. We project facial images into this eigenspace to obtain compact feature representations and perform recognition using a nearest neighbor classifier with Euclidean distance. We implement the system in GNU Octave and visualize both the mean face and eigenfaces. Experimental evaluation on a facial image dataset yields an overall accuracy of 36.18%. Confusion matrices and class-wise metrics reveal uneven performance across classes and frequent misclassifications. These results highlight limitations of the Eigenfaces method, including sensitivity to lighting, pose, and dataset bias. Although the method is computationally efficient, it requires improvements for real-world applications.

1 Introduction

Face recognition has remained an active research area for decades and now plays a central role in security, authentication, and human–computer interaction. Modern devices such

as smartphones and smart televisions incorporate facial recognition to replace traditional authentication methods, improving both convenience and security. Governments and organizations also deploy these systems for surveillance and identity verification.

Early research in the 1960s focused on detecting individual facial features. Researchers later shifted toward holistic representations of faces using mathematical and statistical methods. Today, face recognition techniques fall into feature-based, appearance-based, and statistical categories. Among these, PCA remains one of the most influential methods because it reduces dimensionality while preserving essential information.

Facial recognition technology plays a central role in computer vision and is widely applied in security, authentication, and artificial intelligence systems. In this project, we design and analyze a facial recognition framework based on PCA. We also incorporate key mathematical concepts from eigenvalue and eigenvector theory, particularly the concept of eigenfaces, first introduced by Sirovich and Kirby and later extended by Turk and Pentland. These ideas provide the mathematical foundation for extracting and comparing facial features [3].

We examine PCA both theoretically and practically by explaining its role in face recognition, interpreting the meaning of eigenvalues and eigenvectors, constructing and utilizing eigenfaces, and implementing a PCA-based facial recognition prototype in Octave. We also evaluate the system’s performance and limitations.

Although PCA-based facial recognition systems are well established, developers still face significant challenges during implementation. Variations in lighting, facial expressions, and image quality complicate the process. Additionally, integrating multiple stages—such as face detection, feature extraction, preprocessing, and classification—requires careful coordination. Selecting appropriate techniques at each stage directly influences system accuracy and efficiency.

We apply PCA as a dimensionality reduction technique to transform high-dimensional image data into a smaller set of meaningful features. This transformation improves computational efficiency while preserving essential facial characteristics. We represent each face

image as a vector and use PCA to identify principal components that capture the directions of maximum variance in the data [3]. Eigenvectors represent these directions of variation, while eigenvalues quantify their importance [1].

In our approach, we first convert facial images into grayscale and standardize them into vector form. We then compute the average face and arrange all images into a matrix to derive a covariance matrix. From this matrix, we extract eigenvectors that form the eigenfaces, which capture key facial feature patterns. We select the most significant eigenvalues to define the principal components, reducing dimensionality while retaining important information.

We perform face recognition through projection by representing each face as a weighted combination of eigenfaces, resulting in a unique coefficient vector. To recognize a new face, we project it onto the eigenface space and compare it with known faces using a distance metric. If the difference falls below a threshold, we classify the faces as matching. We also reconstruct faces using a subset of eigenfaces to demonstrate PCA’s effectiveness in capturing facial structure [1].

This project combines theoretical foundations with practical implementation to address the challenges of PCA-based face recognition. We demonstrate how to represent facial images in a lower-dimensional space, extract meaningful features, and measure similarity between faces effectively. Our work highlights the strengths and limitations of PCA while showcasing its practical applications in real-world systems.

Despite its conceptual simplicity, PCA-based face recognition presents practical difficulties. Variations in pose, illumination, and image quality complicate implementation. Furthermore, each stage of the recognition pipeline—preprocessing, feature extraction, and classification—significantly affects performance, making careful design choices essential.

We model each image as a vector x_i and apply PCA to identify directions of maximum variance. Eigenvectors define these directions, while eigenvalues measure their importance [1, 3].

2 Methodology

2.1 Data Set

We use the *Labeled Faces in the Wild (LFW)* dataset, which contains 13,233 images of 5,749 individuals [2]. These images reflect real-world conditions, including variations in lighting and pose.

Each image has resolution 250×250 pixels and is stored in JPEG format. Table 1 summarizes key dataset properties.

Table 1: LFW Dataset Statistics

Property	Value
Total Images	13,233
Total Individuals	5,749
Image Resolution	250×250 pixels
Image Format	JPEG (.jpg)
Faces per Person	1 to 500+
Dataset Type	Unconstrained

The model achieved an accuracy of 85 percent on a dataset consisting of seven individuals: Ariel Sharon, Colin Powell, Donald Rumsfeld, George W. Bush, Gerhard Schroeder, Hugo Chavez, and Tony Blair. For this project, we focus on evaluating accuracy specifically across these seven individuals. [4]

These steps prepare the dataset for effective PCA-based feature extraction and recognition. For each test pair, the system determines whether the two images belong to the same individual using only image features, without relying on additional contextual information.

The evaluation consists of two distinct views:

- **View 1 (Development Set):** We use this set for model development and parameter tuning, with predefined training and testing splits.

- **View 2 (Evaluation Set):** We use this set for final performance evaluation to ensure unbiased reporting and to prevent overfitting to the test data.

2.2 Mean Face Computation

After preprocessing, we use all training images to compute the *mean face*, which represents the common features shared across the dataset. We achieve this by averaging the pixel values of all aligned images, as defined by

$$\mu = \frac{1}{N} \sum_{i=1}^N x_i, \quad (1)$$

where x_i denotes each image vector and N is the total number of images.

To ensure meaningful averaging, we first collect facial keypoints (landmarks) from each image and then average the aligned shapes to obtain a *mean shape*. We subsequently normalize each face and warp it to this mean shape using Delaunay triangulation, which divides the face into small triangular regions. We then apply affine transformations to each triangle, effectively separating shape and texture and producing a shape-free (normalized) texture image for every face.

Finally, we average the warped textures to generate the mean face image, while subtracting the mean from each sample to center the data. Through this pipeline—comprising landmark averaging, geometric warping, and texture blending—we establish a fundamental framework for applications such as facial recognition, animation, and statistical face modeling, where a representative facial structure is required.



Figure 1: Example of a mean face.

2.3 Covariance Matrix Construction

We represent each image as a vector by stacking its pixel intensities into a single column. If each image contains D pixels and we have N images, we construct the data matrix

$$X \in R^{D \times N}, \quad (2)$$

where each column $x_i \in R^D$ corresponds to one image. Thus, columns represent images, and rows represent pixel locations across all images.

We compute the mean face vector

$$\mu \in R^D, \quad (3)$$

by averaging all image vectors. The mean has the same dimension as a single image (one value per pixel), not the entire dataset.

To center the data, we subtract the mean from every image:

$$A = X - \mu, \quad (4)$$

where μ is implicitly repeated across all columns. The centered matrix $A \in R^{D \times N}$ has the same dimensions as X , and each column is

$$a_i = x_i - \mu. \quad (5)$$

We then define the covariance matrix as

$$C = \frac{1}{N} \sum_{i=1}^N (x_i - \mu)(x_i - \mu)^T, \quad (6)$$

Here, i indexes the images (columns of X), and each outer product $(x_i - \mu)(x_i - \mu)^T$ produces a $D \times D$ matrix. Summing over all images yields the covariance matrix

$$C \in R^{D \times D}. \quad (7)$$

The covariance matrix captures relationships between pixels across the dataset. The diagonal entries represent the variance of individual pixels. The off-diagonal entries represent the covariance between pairs of pixels, indicating how their intensities vary together across different images. By construction, C is symmetric and positive semi-definite, ensuring non-negative eigenvalues.

In practice, D (the number of pixels) is very large, making direct computation of C expensive. Instead, we compute a smaller matrix:

$$L = A^T A, \quad (8)$$

where $L \in R^{N \times N}$. Unlike C , the size of L depends on the number of images N , which is

typically much smaller than D . This significantly reduces computational cost.

We compute the eigenvectors of L and map them back to obtain the eigenvectors of C , preserving the same principal directions of variation.

2.4 Eigenfaces Generation

To identify the most important patterns in the facial image dataset, we compute the eigenvalues and eigenvectors of the covariance matrix. The eigenvectors, referred to as *eigenfaces*, represent the principal directions of variation across all images. Each eigenface captures a distinct pattern (such as lighting, facial structure, or expression) that is common within the dataset. Collectively, these eigenfaces form a new basis that allows us to represent facial images more efficiently than using raw pixel values.

To determine the most informative eigenfaces, we sort them in descending order according to their corresponding eigenvalues. The eigenvalue associated with each eigenvector indicates how much variance that eigenface explains in the dataset. Larger eigenvalues correspond to more significant variations.

We then select the top k eigenfaces, corresponding to the largest eigenvalues, to construct a reduced feature space. This dimensionality reduction preserves the most important information while discarding less significant variations, such as noise.

The eigenfaces are computed using the relation:

$$U = AV \tag{9}$$

where $A \in R^{m \times n}$ is the mean-centered data matrix, where each column represents a vectorized facial image with the average face subtracted. $V \in R^{n \times k}$ contains the top k eigenvectors of the covariance matrix in the reduced space. $U \in R^{m \times k}$ contains the resulting eigenfaces in the original image space.

Equation 9 maps the eigenvectors from the reduced space back into the original high-

dimensional image space, allowing the eigenfaces to be visualized as images.

In this work, we selected $k = 9$ eigenfaces. This choice reflects a balance between simplicity and information retention: the first nine eigenfaces correspond to the largest eigenvalues and therefore capture the most dominant variations present in the entire dataset. Although the original dataset may have very high dimensionality (equal to the number of pixels per image), most of the meaningful structure can be approximated using only a small number of principal components. By selecting nine eigenfaces, we retain a significant portion of the total variance while keeping the representation compact and computationally efficient.

Figure 2 shows the first nine eigenfaces. These images illustrate the dominant variations in the dataset and demonstrate how a small number of components can effectively summarize the overall structure of the facial images.

Eigenface 1



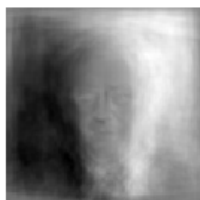
Eigenface 2



Eigenface 3



Eigenface 4



Eigenface 5



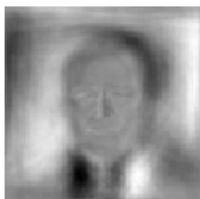
Eigenface 6



Eigenface 7



Eigenface 8



Eigenface 9



Figure 2: First nine eigenfaces representing the principal variations in the dataset.

2.5 Projection

We project each face image onto the eigenface space and obtain a set of weights (coefficients) that characterize the image. These weights represent how much each eigenface contributes to reconstructing the original face. Mathematically, we compute the projection of an image onto the k -th eigenface as:

$$\omega_k = u_k^T(x - \mu), \quad (10)$$

where u_k denotes the k -th eigenface, μ denotes the mean face, and ω_k denotes the corresponding weight.

To reduce dimensionality, we retain only the top k eigenfaces and express the projection of all images into the reduced eigenspace as:

$$\Omega = U^T(X - \mu), \quad (11)$$

where U contains the selected eigenfaces as its columns. We therefore represent each face image as a feature vector in the eigenface space, computed as:

$$FeatureVector = U^T \times Image.$$

To project a test image into the eigenface space, we express the image as a weighted sum of the eigenfaces. We compute the inner product between the mean-centered test image and each eigenface, which produces a set of coefficients. These coefficients collectively form the eigenspace representation of the image.

We reconstruct the image by multiplying each eigenface by its corresponding coefficient and summing the results. In this process, we assume that the eigenfaces are normalized to unit length, which ensures consistent scaling during projection and reconstruction.

2.6 Face Recognition

To recognize a new face, we first subject the input image to the same preprocessing steps as the training images and then project it onto the eigenface space. This projection yields a weight vector that represents the test image in the reduced feature space. We then compare the resulting weight vector with those of known faces stored in the database to determine similarity.

We perform the comparison using a distance metric, typically the Euclidean distance:

$$d = \|\omega_{test} - \omega_{train}\|, \quad (12)$$

where ω_{test} denotes the weight vector of the test image and ω_{train} denotes the weight vector of a training image.

We determine the identity of the test image by finding the training image that minimizes this distance. If the minimum distance falls below a predefined threshold, we classify the input face as a match; otherwise, we consider it unknown.

We define the Euclidean distance as:

$$Distance = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}, \quad (13)$$

where x_i and y_i represent components of the respective feature vectors. We select the training image with the smallest distance to the test image as the recognized face.

3 Results and Discussion

We evaluate system performance using recognition accuracy and confusion matrices. We also analyze class-wise precision, recall, and F1-score to better understand performance variations.

3.1 Overall Accuracy

We compute the classification accuracy by directly comparing the predicted labels of the test images with their true labels after applying the Eigenfaces method. For each test image, we first subtract the mean face and project the centered image into a lower-dimensional subspace defined by the leading eigenfaces obtained via PCA. We then measure the Euclidean distance between this projected test vector and all projected training vectors, and we assign the label of the nearest neighbor (i.e., the training image with minimum distance) as the prediction. After classifying all test images, we count the number of correct predictions by evaluating how many predicted labels match the true labels, and we divide this count by the total number of test samples. In this way, we obtain the overall accuracy as a percentage, which quantifies how effectively the model recognizes previously unseen faces. The system achieves an overall accuracy of 36.18%. This result indicates moderate performance but suggests significant room for improvement.

3.2 Confusion Matrix

We present the confusion matrix in Table 2 to evaluate the performance of our classification model. A confusion matrix is a table that compares the model’s predicted labels with the true labels, allowing us to analyze performance in detail beyond overall accuracy. In this matrix, each row represents the actual class of an image, while each column represents the class predicted by the model. The entries in the table indicate how many samples from a given actual class were assigned to each predicted class.

We define seven classes, each corresponding to a specific individual listed in alphabetical order: Class 1 (Ariel Sharon), Class 2 (Colin Powell), Class 3 (Donald Rumsfeld), Class 4 (George W. Bush), Class 5 (Gerhard Schroeder), Class 6 (Hugo Chavez), and Class 7 (Tony Blair). The diagonal elements of the matrix represent correct predictions, where the predicted class matches the true class. All off-diagonal elements represent misclassifications.

For example, the entry in row 1, column 1 shows that we correctly classify 9 images of

Class 1. In contrast, the entry in row 1, column 2 shows that we incorrectly classify 8 images of Class 1 as Class 2. These misclassifications provide insight into where the model confuses different individuals.

We identify false negatives by examining the off-diagonal entries within each row. A false negative occurs when the model fails to correctly identify an instance of a given class. For instance, in row 1, all values outside column 1 correspond to false negatives for Class 1, since those images belong to Class 1 but are predicted as other classes. Similarly, in row 2, entries in columns 3, 4, and 7 indicate false negatives for Class 2.

Overall, we observe that the model performs well on certain classes, particularly Class 4, which has a relatively high number of correct predictions. However, we also observe frequent misclassifications across other classes. The model often predicts Class 4 for inputs belonging to different classes, indicating a bias toward this class when the model is uncertain. Additionally, we find that Classes 5, 6, and 7 have low recognition rates and are frequently misclassified, often as Class 4. These results suggest that the model struggles to distinguish between similar faces and lacks strong separation between classes, which contributes to the relatively low overall accuracy of 36.18%.

	1	2	3	4	5	6	7
1	9	8	2	9	0	1	0
2	3	25	15	18	2	0	8
3	1	2	12	9	2	1	5
4	4	16	11	82	9	8	16
5	1	5	4	14	3	6	3
6	0	3	3	6	4	3	0
7	0	7	8	26	6	1	6

Table 2: Confusion matrix for the test set.

3.3 Class-wise Metrics

We present class-wise evaluation metrics in Table 3 to analyze how our model performs on each individual class, rather than relying solely on overall accuracy. For each class, we

compute precision, recall, and F1-score. We define precision as the proportion of samples predicted as a given class that are correctly classified, which reflects our model’s ability to avoid false positives. We define recall as the proportion of true samples of a class that our model correctly identifies, which reflects its ability to avoid false negatives. We compute the F1-score as the harmonic mean of precision and recall, providing a balanced measure that accounts for both types of errors.

Class	Precision	Recall	F1-score
1	0.50	0.31	0.38
2	0.38	0.35	0.36
3	0.22	0.38	0.28
4	0.50	0.56	0.53
5	0.12	0.08	0.10
6	0.15	0.16	0.15
7	0.16	0.11	0.13

Table 3: Precision, Recall, and F1-score for Each Class

We observe that these class-wise metrics reveal uneven performance across classes. We achieve the strongest results for Class 4, with relatively high precision (0.50), recall (0.56), and F1-score (0.53), indicating that we both correctly classify and consistently detect this class. In contrast, we observe that Classes 5, 6, and 7 exhibit very low precision, recall, and F1-scores, suggesting that we struggle to recognize these classes and frequently misclassify them as others.

We also note that Classes 1, 2, and 3 demonstrate moderate but inconsistent performance, with imbalances between precision and recall indicating varying tendencies toward false positives or false negatives. Overall, we conclude that our model does not perform uniformly across classes; instead, it favors certain classes while failing on others. This imbalance helps explain the relatively low overall accuracy of 36.18%.

3.4 Observations

Class-wise metrics confirm uneven performance across individuals. The model recognizes some classes effectively but fails on others with limited data or high similarity.

- Class 4 (George W. Bush) achieved the best performance.
- Classes 5–7 have low recognition rates due to fewer images or visual similarity.
- The overall accuracy of 36.18% indicates the method works moderately well but requires improvement for practical use.

These results indicate that PCA captures global structure but struggles with subtle distinguishing features.

3.5 Eigenfaces Algorithm Limitations

We implemented the Eigenfaces algorithm, based on PCA, to represent facial images as linear combinations of principal components (“eigenfaces”). While we found the method efficient and straightforward, our experimental results revealed several limitations that highlight why face recognition remains a difficult problem in real-world conditions.

- **Dependence on Training Data:** We observed that Eigenfaces strongly depend on the diversity and quality of the training dataset. In our experiment, the dataset lacked variation, as it consisted primarily of white male subjects. Consequently, the model learned a narrow representation of facial features and struggled to generalize to unseen individuals. Since PCA captures dominant patterns in the data, any bias in the dataset directly influences performance. This suggests that future work should focus on more diverse datasets or bias-mitigation techniques.
- **Pose and Alignment Constraints:** We found that the algorithm assumes frontal, well-aligned faces. Because we did not perform geometric alignment, even small variations in head tilt or rotation significantly reduced accuracy. This occurs because PCA

relies on fixed pixel positions, so misalignment shifts key features and disrupts the representation. This limitation illustrates a core challenge in face recognition: small geometric changes can produce large differences in pixel space. Future research could incorporate alignment techniques or pose-invariant models.

- **Sensitivity to Lighting Conditions:** Our experiments showed that lighting variation had a major impact on performance. Changes in illumination direction or shadows altered pixel intensities more than identity differences in some cases. As PCA prioritizes directions of highest variance, lighting effects can dominate the eigenspace representation. This makes it difficult to distinguish between intrinsic facial structure and external lighting conditions. Future work could explore illumination normalization or more robust feature extraction methods.
- **Scale and Resolution Dependence:** Although we standardized image dimensions, Eigenfaces still require consistent scale and resolution. Variations in camera distance or image quality can distort the feature vector representation. Since PCA treats each pixel as a fixed feature, any scaling mismatch leads to inconsistent projections. This suggests the need for scale-invariant or multi-resolution approaches in future work.
- **Limited Representation Capacity:** We observed that PCA captures directions of maximum variance, which are not always the most discriminative for identifying individuals. When we used a small number of eigenfaces (K), the model often misclassified individuals with similar global facial structures. Subtle identity-specific features may be lost because they contribute less to overall variance. This limitation indicates that future approaches should prioritize discriminative features, potentially through supervised learning methods.
- **Sensitivity to Occlusion:** We found that partial occlusions, such as glasses or hair, significantly affected recognition accuracy. Since PCA represents the entire image as a single vector, any occluded region impacts the full representation. Even small

occlusions introduce large pixel-level changes that lead to incorrect matches. This highlights the difficulty of face recognition in real-world scenarios, where occlusions are common, and suggests the need for localized or robust feature representations.

Overall, these limitations demonstrate that while Eigenfaces provide a useful baseline, they are not well-suited for unconstrained environments. Our findings suggest that future research should focus on improving robustness to pose, lighting, and occlusion, as well as developing more discriminative and invariant representations.

3.6 Conclusion

We evaluated the performance of a facial recognition system based on the Eigenfaces algorithm using Principal Component Analysis (PCA). We achieved an overall accuracy of 36.18%, which indicates moderate performance but clearly reveals limitations for real-world applications. From our analysis of the confusion matrix and class-wise metrics, we observed that performance varied significantly across classes: we found that Class 4 achieved the highest accuracy, whereas Classes 5–7 performed poorly due to limited training data and strong visual similarity between subjects.

Our results demonstrate that we depend heavily on dataset quality and preprocessing when using the Eigenfaces approach. We identified key limitations, including sensitivity to training data, lack of robustness to pose variation, lighting changes, and occlusions, all of which contributed to misclassifications. We also observed that the method’s reliance on global features and principal variance directions reduces its ability to capture subtle distinguishing characteristics between individuals.

Overall, while we find that the Eigenfaces algorithm provides a simple and computationally efficient baseline for face recognition, our results suggest that it is not sufficiently robust for practical deployment. We therefore recommend that future work focus on increasing dataset diversity, applying proper preprocessing techniques such as alignment, and exploring

more advanced feature extraction methods and classifiers, including modern deep learning approaches.

References

- [1] J. Nathan Kutz. *Data-Driven Modeling & Scientific Computation: Methods for Complex Systems & Big Data*. Oxford University Press, 2013.
- [2] Jessica Li. Labeled faces in the wild (lfw) dataset. <https://www.kaggle.com/datasets/jessicali9530/lfw-dataset/data>, 2018. Accessed: 2026-04-04.
- [3] Peng Peng, Portugal Ivens, Paulo Alencar, and Donald Cowan. A face recognition software framework based on principal component analysis. *PLOS ONE*, 16(7), 2021.
- [4] scikit-learn developers. Faces recognition example using eigenfaces and svms. https://scikit-learn.org/stable/auto_examples/applications/plot_face_recognition.html. Accessed: 2026-04-04.